



Today's agenda

- ↳ BST to greater sum
- ↳ Inorder successor in BST
- ↳ Populate next Pointer on right.
- ↳ Path from root to given Node.
- ↳ LCA in Binary tree
- ↳ LCA in BST.



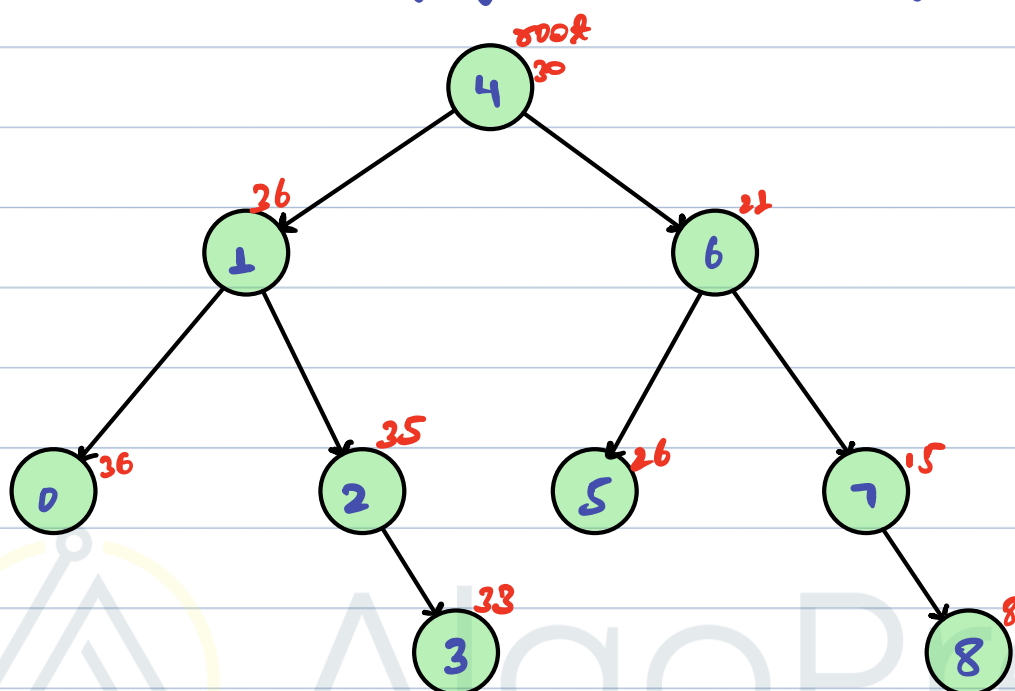
AlgoPrep

{leetcode 1038}

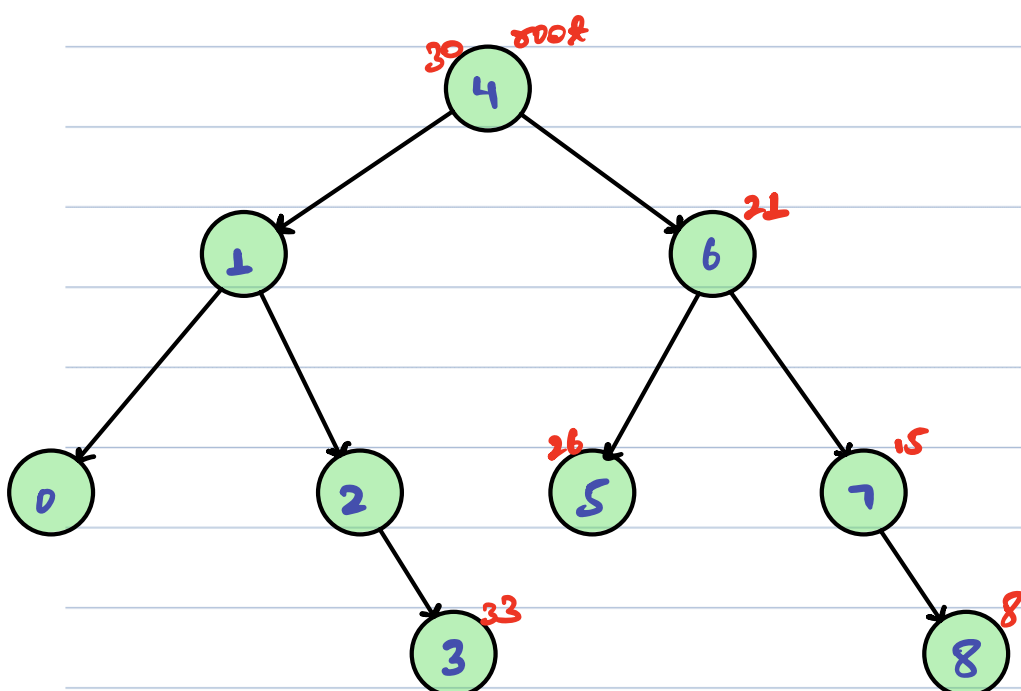


Q) BST to greater sum

↳ Convert every node of BST to the original key plus the sum of all keys greater than the original key in BST.

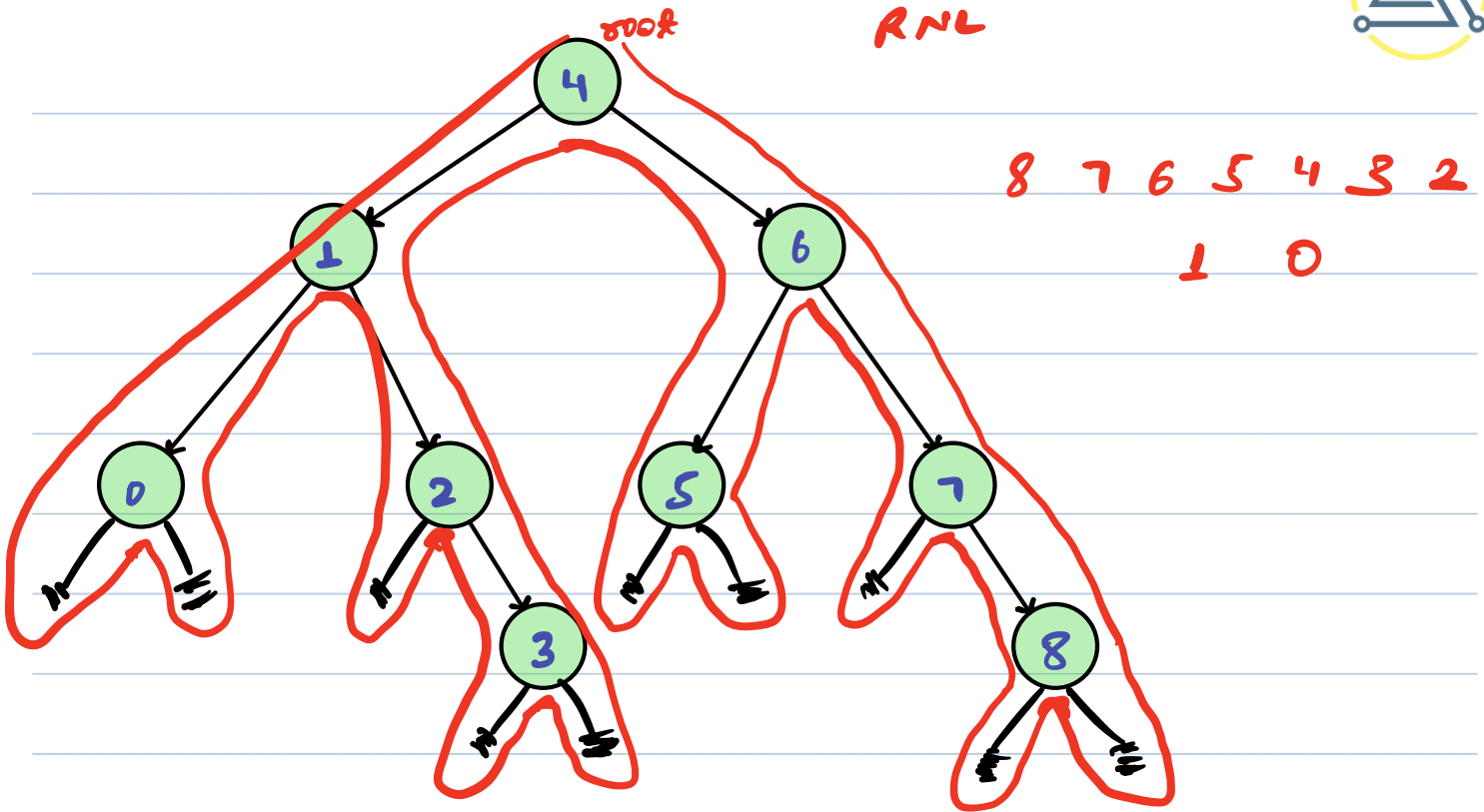


→ inorder: Sorted in inc. order → LNR



$$\begin{aligned} \text{greater} &= 0+8+7 = 15+6 = 21 \\ &= 21+5 = 26+4 = 30+3 \\ &= 33 \end{aligned}$$

→ Reverse inorder: Sorted in dec. order → RNL



// Pseudo code

```
Public Node SumtoGreater (Node root) {  
    helper (root);  
    return root;  
}
```

```
int greater = 0;
```

```
Public void helper (Node root) {  
    if (root == null) { return; }
```

```
    helper (root.right);
```

```
    greater = greater + root.val;
```

```
    root.val = greater;
```

```
    helper (root.left);  
}
```

T.C: $O(N)$

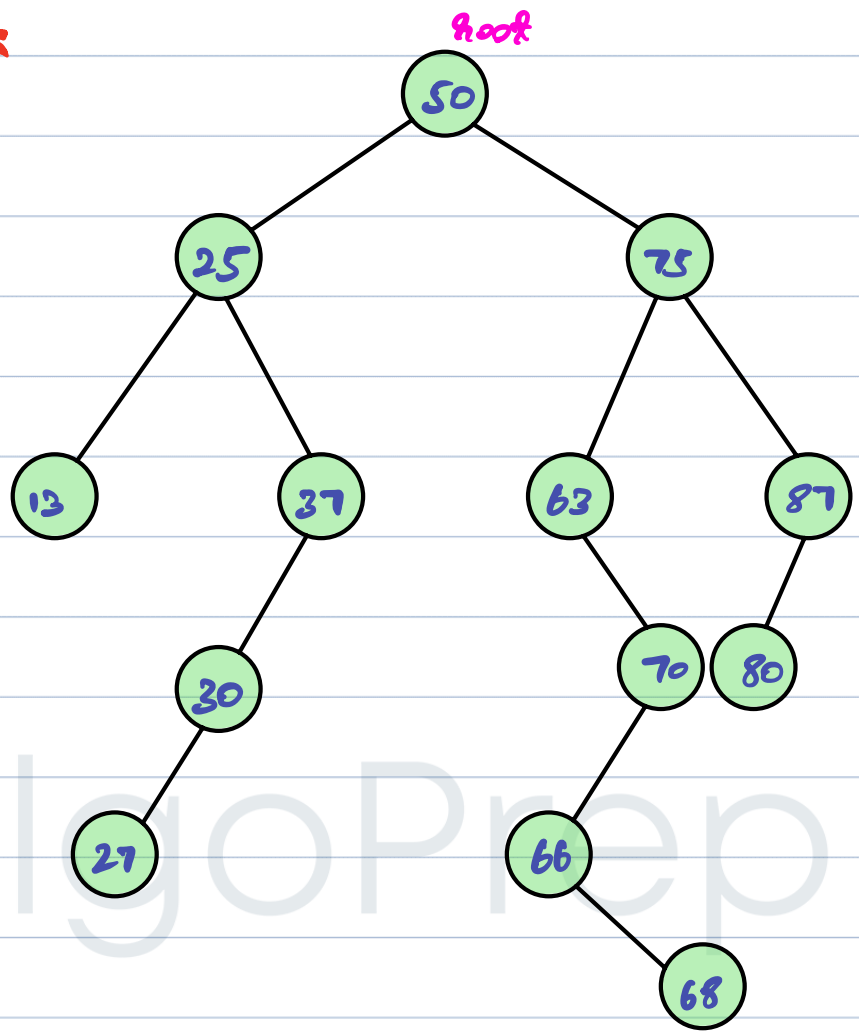
S.C: $O(H)$



Q) Inorder Successor of K in BST

↓
Just greater to K

IS(68) : 70



Idea 1

↳ inorder traversal → sorted value

↳ The first val $> K$ is your ans.

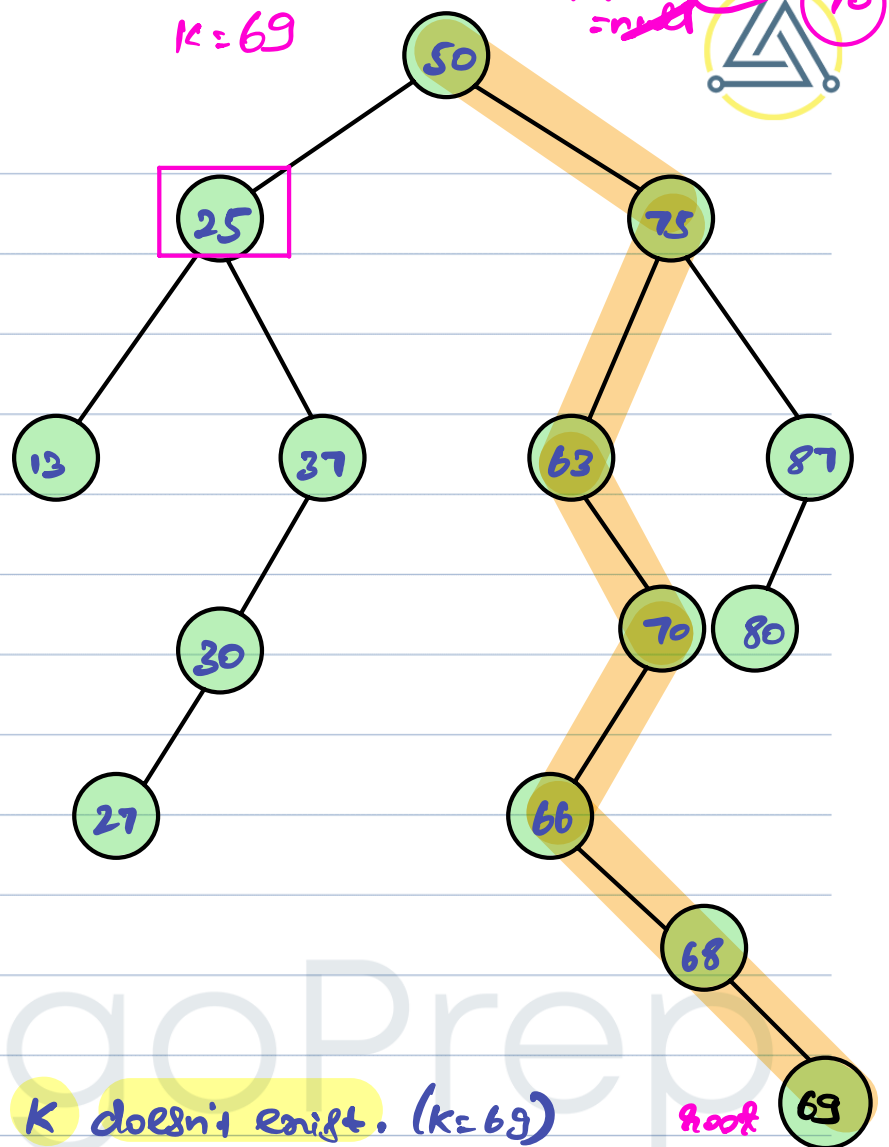
T.C: $O(N)$

Obs 1:

① if right subtree of k exists.



ans = leftmost node of
that subtree.



obs2: if right subtree of K doesn't exist. ($K=69$)

Ans: last Node from where you taking left turn.



Iterative code

```
Node inorderSuccessor (Node root, int k)
```

```
Node temp = null;
```

```
while (true) {
```

```
    if (root.val > k) {  
        temp = root;  
        root = root.left;
```

```
    }
```

```
    else if (root.val < k) {  
        root = root.right;
```

```
    }
```

```
    else
```

```
        break;
```

```
    }
```

```
}
```

```
if (root.right == null) {  
    return temp;
```

```
}
```

```
Node rootP = root.right;
```

```
while (rootP.left != null) {  
    rootP = rootP.left;
```

```
}
```

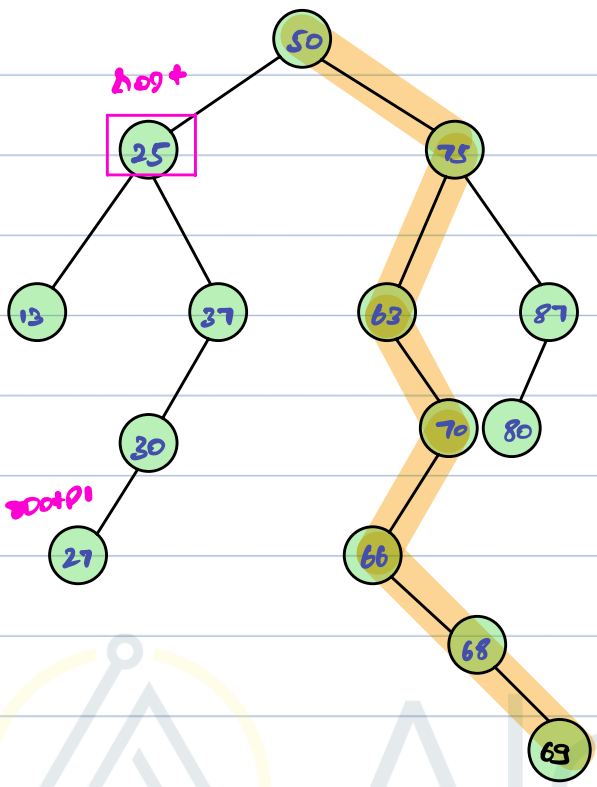
```
return rootP;
```

T.C: $O(H)$
S.C: $O(1)$



$k=25$

temp = null (50)



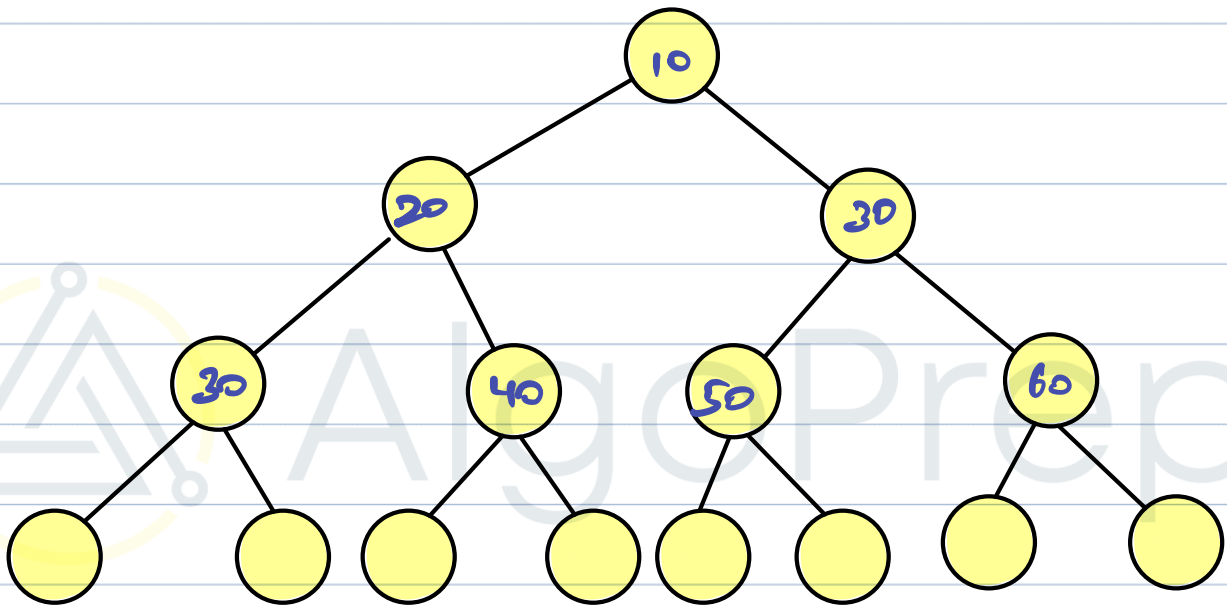
AlgoPrep



* Perfect binary tree

① Each node can have 0 or 2 child nodes.

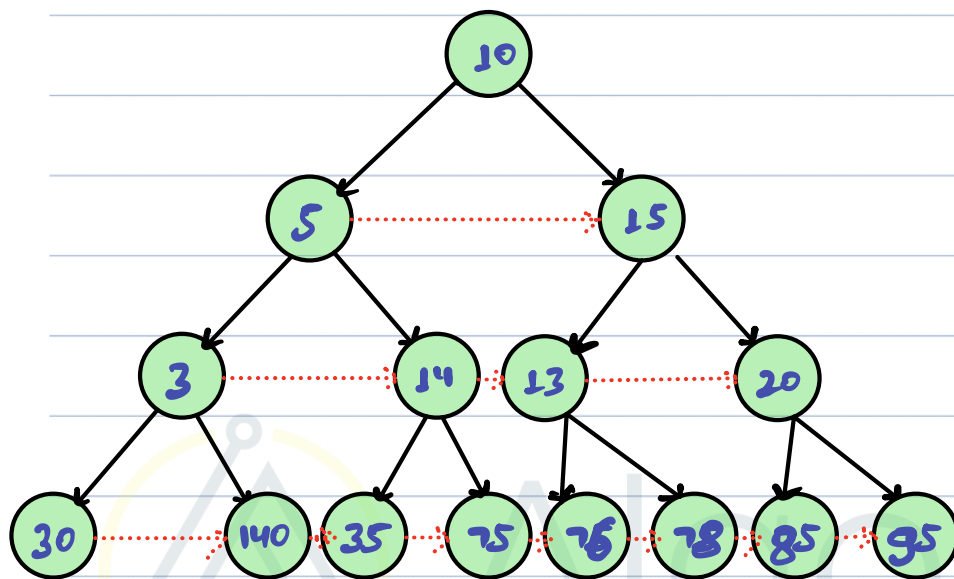
② The child nodes will be added in order.





Q) Populate next pointer

↳ Given a Perfect binary tree, Point next pointers to the node on right side.



```
class Node {  
    int val;  
    Node left;  
    Node right;  
    Node next;
```

}

Idea 1

↳ do level order and set the next pointers.

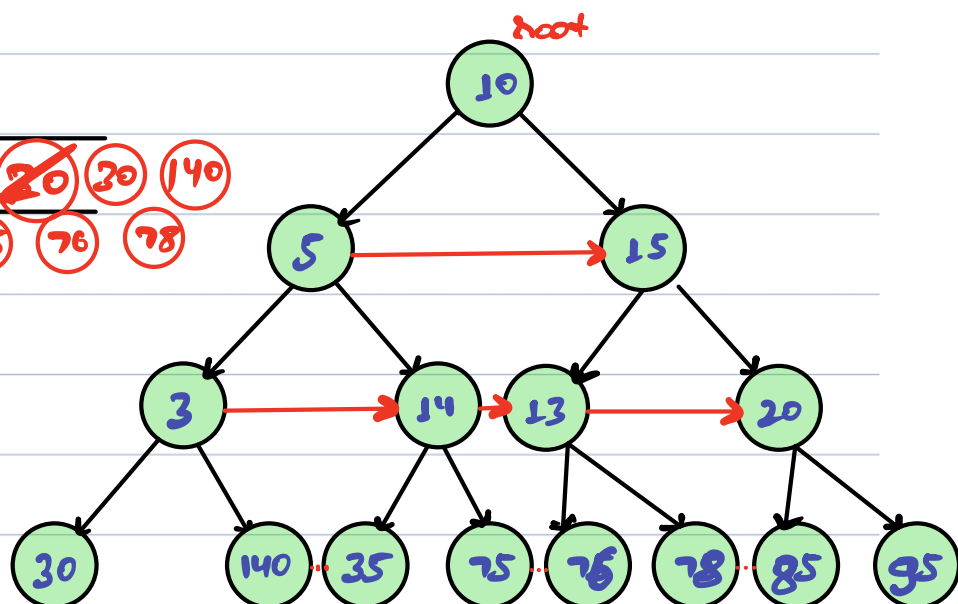
size = 4

q:



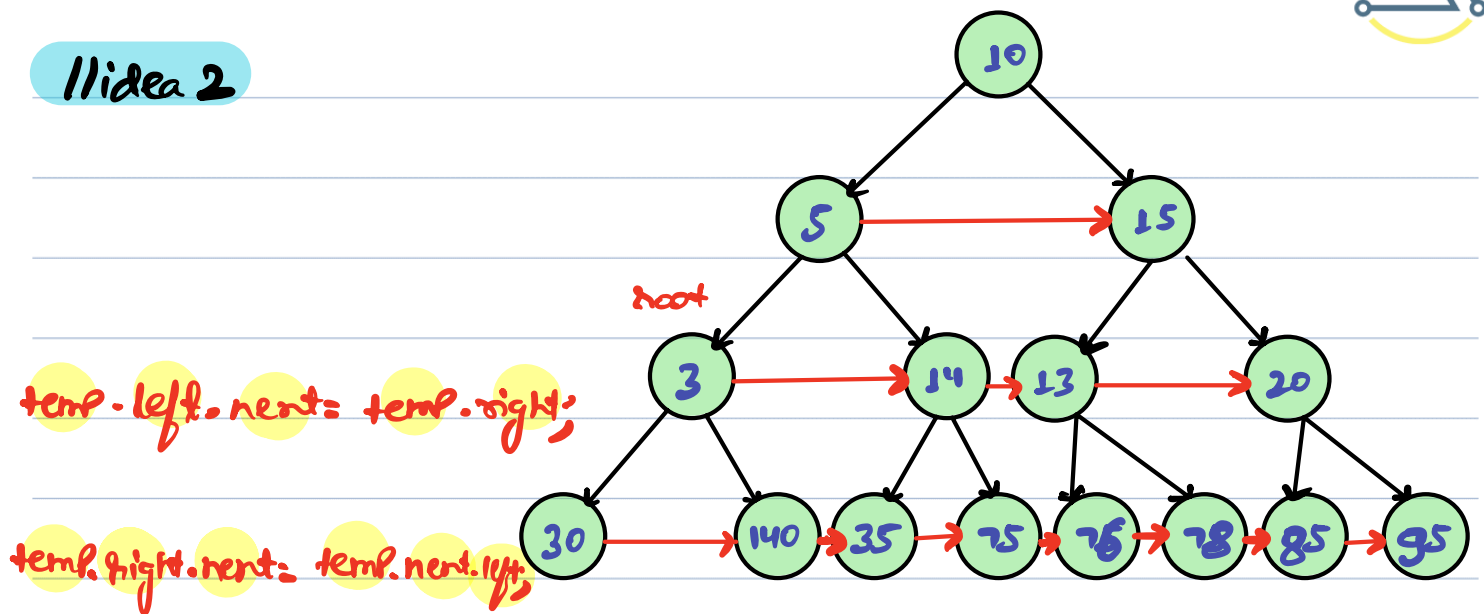
rem = 20

T.C: $O(N)$ S.C: $O(1)$





Idea 2



$temp = temp \cdot next;$

Pseudo code

```
void NextPointer (Node root) {  
    while (root.left != null) {  
        Node temp = root;  
        while (temp != null) {  
            temp.left.next = temp.right;  
            if (temp.next != null) {  
                temp.right.next = temp.next.left;  
            }  
            temp = temp.next;  
        }  
        root = root.left;  
    }  
}
```

T.C: $O(N)$
S.C: $O(1)$



note: we allow a node to be ancestor to itself.



$LCA(J, I) = C$

$LCA(L, D): D$

1/idea 1

↳ $LCA(m, n) = c$

Path from root to m: A B C D E m

Path from root to H: A B C G H



// Pseudo Code

```
Node LCA (Node root, int n1, int n2) {  
    if (root == null) { return null; }  
    if (root.val == n1 || root.val == n2) {  
        return root;  
    }  
}
```

```
Node l = LCA (root.left, n1, n2);
```

```
Node r = LCA (root.right, n1, n2);
```

```
if (l != null && r != null) {  
    return root;  
}
```

```
}
```

```
if (l != null && r == null) { return l; }
```

```
if (r != null && l == null) { return r; }
```

```
return null;
```

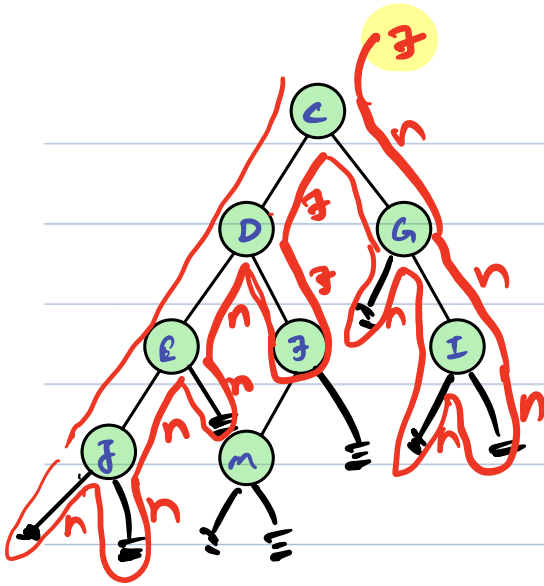
```
}
```

T.C: $O(N)$

S.C: $O(H)$



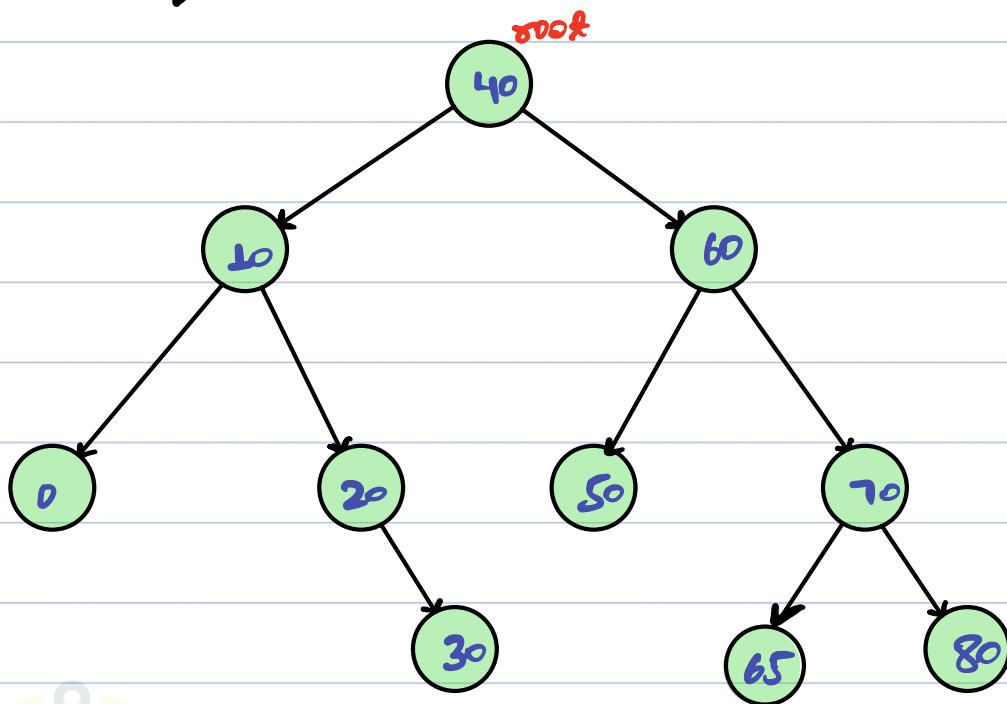
$LCA(7, m) : 7$



AlgoPrep



Q) LCA of BST



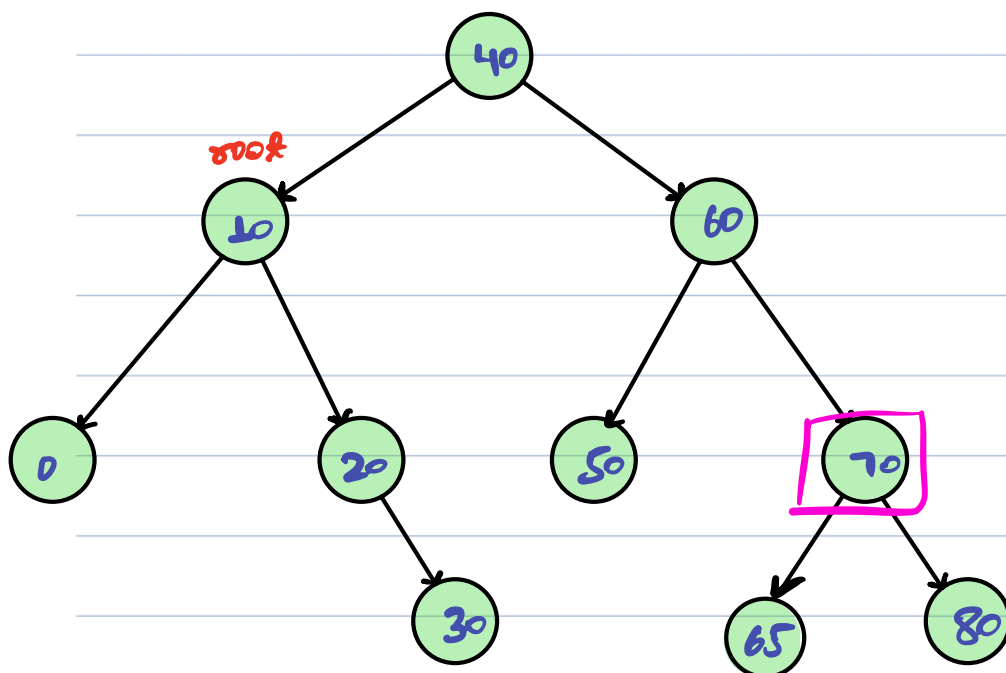
//idea 1

↳ run the same code as BT.

T.C: $O(N)$

S.C: $O(H)$

//idea 2



$LCA(65, 80)$

↓
70

$LCA(10, 30)$

↓
10



//Pseudo code

```
int LCAinBST (Node root, int n1, int n2){  
    while (true){  
        if (n1 < root.val && n2 < root.val){  
            root = root.left;  
        }  
        else if (n1 > root.val && n2 > root.val){  
            root = root.right;  
        }  
        else {  
            return root.val;  
        }  
    }  
}
```

T.C: $O(H)$

S.C: $O(1)$